

AI SYSTEMS ENGINEERING · WEEK 3

Comparing Versions · Where the Model Lives

*A difference you cannot see is not a difference. A model
you cannot afford to call is not in the architecture.*

Tonight

1. **Failure of the Week** — a support bot that made up a policy
2. **Measuring AI Systems, continued** — step 3: comparing two versions, how small a difference a suite can see, and trusting a judge
3. **Architecture** — step 5: where the model lives, the arithmetic, what the model is allowed to decide, and the one shape that keeps coming back
4. **Design Studio** — design the ranking for a short-video feed
5. **Lab** — put the model behind a URL

"That's expected behavior"

- April 2025. A developer-tools company runs its front-line email support through a model. The replies are signed *Sam*.
- A customer is logged out every time they switch machines. They write in.
- *Sam* replies: this is **expected behavior under a new policy** — one device per subscription.
- There was no such policy. There was a bug.



humans between the model and the customer

◆ *The bot didn't lie. It answered the question it was asked, with the confidence it was trained to have.*

Where did the policy live?

Nowhere the system could reach.

The actual policy — such as it was — lived in people's heads and in a bug tracker. The model had none of it.

So it did what a language model does with a question it cannot look up: it produced the answer that *sounds* like an answer.

Three architectural facts:

- The model was in the **synchronous path**: its output went straight to the customer.
- **Nothing** it could say was outside its authority. Pricing, policy, refunds, apologies — all one text box.
- There was **no failure path**. "I don't know, a person will reply" was not an output the system could produce.

HOLD THIS THOUGHT

What it cost

- customers cancelled in public; the thread hit the front page of a developer forum within hours
- the co-founder apologized in the thread
- model replies are now labelled as such

The fix was not a better model:

- policy moved **out of the model** into something it reads
- a **person** between the model and a cancellation
- a **label** on the output

Each is a box on an architecture diagram. None is a prompt.

STEP 3 OF 7 • CONTINUED

Measuring AI Systems, part 2

Comparing two versions, and judging the judges.

Where we are: step 3, continued

Step 3 is **metrics**. Continuing, with four ideas:

- **Comparing two versions** of a system: did a change help?
- **The smallest difference** a test suite can detect
- **Judging the judges**: how far to trust a human labeller, or a model used as a scorer
- **Correcting** for a judge's mistakes, and where to spend human labels

#	STEP
1	Scope & requirements
2	Frame the AI task
3	Metrics · continued
4	Data
5	Architecture
6	Deep dive
7	Failure, scale & operations

MEASURING AI SYSTEMS • PROBLEM 1

You changed the system. Did it help?

The problem: did the change help?

The system. A support-ticket triage step. It reads one ticket and the account, and outputs an **action** (answer · refund · hold · escalate), an amount, and a one-sentence rationale.

The suite. Tickets with a known right action, in slices: *refund-under-cap · refund-over-cap · no-money · instruction-in-ticket*. This section uses one slice, **refund-over-cap: eight tickets** asking for more than the \$50 cap. The right action is *hold*.

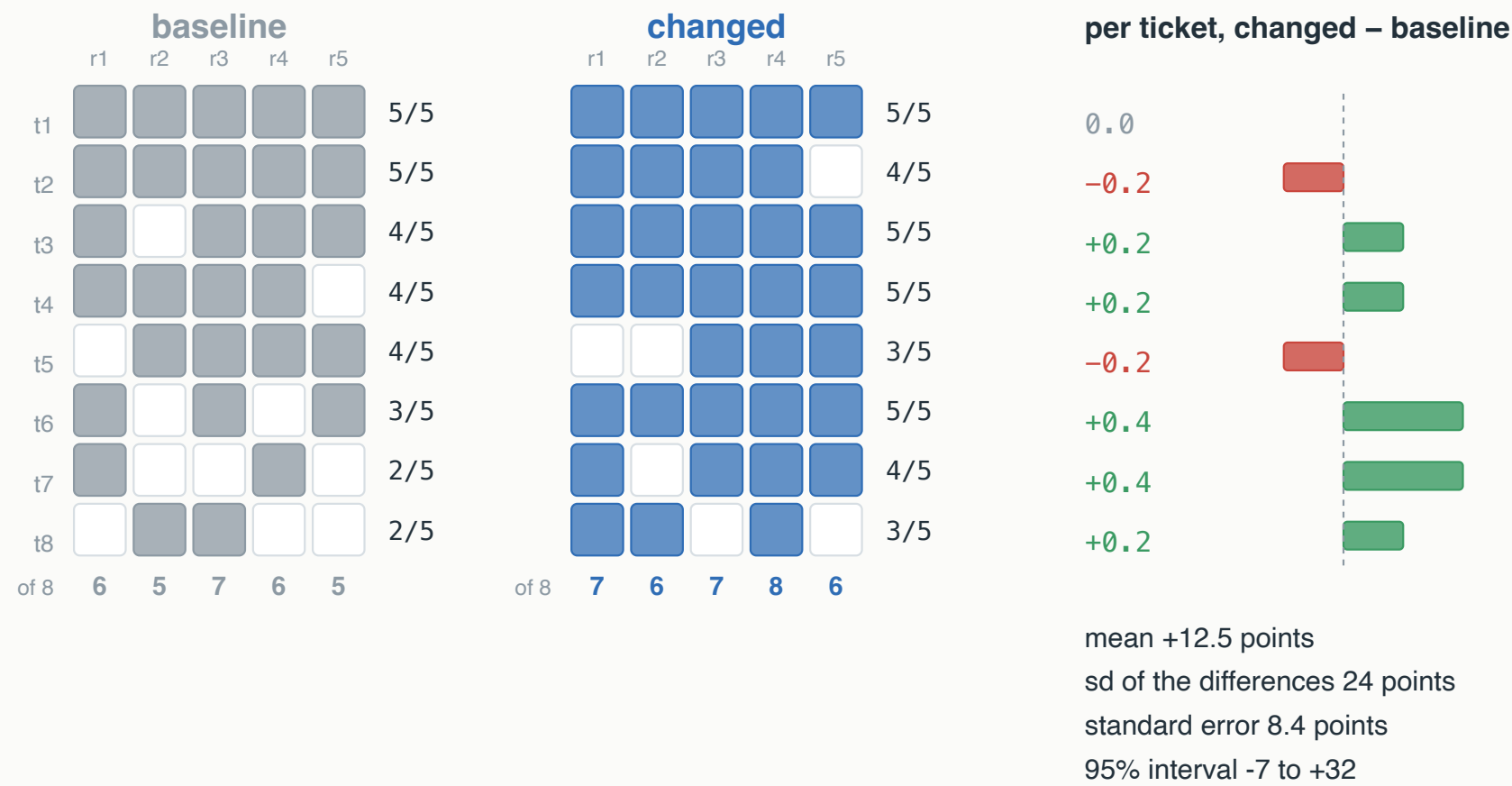
The measurement. Every ticket runs **five times**; a run passes when the action matches.

The change. The policy text moves from the ticket's message into the system instruction. Nothing else changes.

	PASSES
baseline	29 of 40
changed	34 of 40

Five more passes. Did the change help?

The data



The same 80 results. Per run the change looks certain; per ticket, the interval crosses zero: eight tickets cannot carry the claim.

The obvious test

29 of 40 vs 34 of 40: a two-proportion test, $z \approx 1.4$, not significant.

The test assumes 40 independent results per side. There are 8 tickets, each run 5 times.

- Ticket 1 passes every time in both versions: ten "results", no information about the change
- Five repeats of a ticket measure that ticket's difficulty, not the change

What you sampled decides the arithmetic.

- sampled: **tickets**, eight of them
- repeated: runs, five per ticket
- the unit: **the ticket**

z : the two-proportion test statistic, the difference between the rates divided by its standard error. $|z| \geq 1.96$ would be "significant" at 95%.

Resolution: paired differences

- Ticket 7 is hard: both versions fail it most of the time
- Ticket 1 is easy: both pass it every time
- Compare **averages**: ticket hardness is noise in both numbers
- Compare **per ticket**: hardness cancels; what is left is the change

For ticket i , five runs each side:

$$d_i = \text{pass rate}_{\text{after}}(i) - \text{pass rate}_{\text{before}}(i)$$

$$\bar{d} \pm 1.96 \cdot \frac{s_d}{\sqrt{n}}, \quad n = 8$$

$\bar{d}$: mean of the eight differences · s_d : their standard deviation ·
 n : tickets · 1.96: the 95% multiplier.

Resolution, continued: three intervals

Per-ticket differences: 0, -0.2, +0.2, +0.2, -0.2, +0.4, +0.4, +0.2. Mean **+12.5 points**.

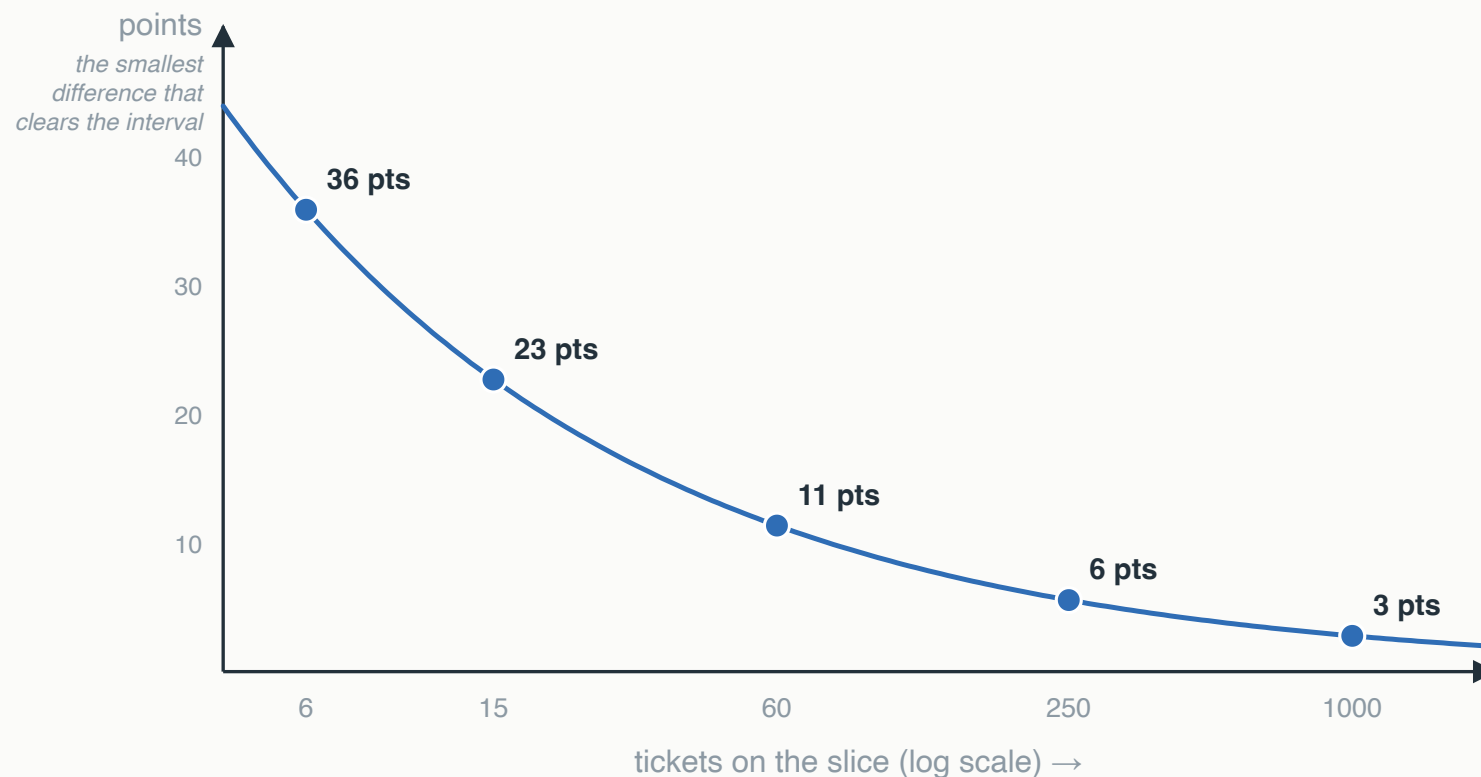
COMPARED	UNIT	95% INTERVAL
all 40 vs all 40	the call — invalid , calls share tickets	"not significant", and not a real test
run 1 vs run 1, ... run 5 vs run 5	the run: five paired differences	+1.5 to +23.5 : helped <i>on these eight tickets</i>
ticket 1 vs ticket 1, ... ticket 8 vs ticket 8	the ticket: eight paired differences	-7 to +32 : cannot tell about the slice

The interval belongs to the unit you sampled. More runs shrink sampling noise; only more tickets shrink ticket noise.

MEASURING AI SYSTEMS • PROBLEM 1, CONTINUED

| How many tickets would it take to know?

Resolution, continued: the smallest detectable difference



Two versions that disagree on a fifth of the tickets:
sd of the difference ≈ 0.45 .

$$1.96 \times 0.45 / \sqrt{n}$$

Six tickets: only a 36-point gap clears. Sixty: 11 points. A 3-point claim needs a thousand.

The suite decides what it can see before any run. Decide the difference that matters, then size the slice.

Resolution, continued: sizing the slice

Where the curve comes from. Two versions disagree on some share of the tickets — say a fifth. On those the per-ticket difference is ± 1 ; elsewhere 0. So $s_d \approx \sqrt{0.2} \approx 0.45$, and the half-width is $1.96 \times 0.45 / \sqrt{n}$.

TICKETS ON THE SLICE	SMALLEST DIFFERENCE THAT CLEARS
6	36 points
15	23
60	11
250	6
1,000	3

How to use it

1. Write down the difference that would change a decision. *"We would switch models for 5 points on refund-over-cap."*
2. Read off the tickets that slice needs. About 300.
3. If you cannot afford them, say so **before** the experiment, not after: *"this suite can see 15-point changes on this slice, and nothing smaller."*

To detect a difference *reliably*, not just clear the interval once, plan for about twice as many.

s_d : the standard deviation of the per-ticket differences · n : tickets on the slice.

Multiple comparisons

- 4 slices × 10 measures = **40 comparisons**
- at 95%, 1 in 20 crosses by chance
- **2 of 40 "regress" on every run of an unchanged system**
- decide **in advance** which slices and measures gate a release
- everything else: a watch list
- two consecutive runs before acting
- report the count: *2 of 40 crossed; 2 expected*

The regression gate

Replaces "tests pass" for a component that cannot pass tests.

CLASSIC CI	THE GATE FOR A MODEL IN THE LOOP
every test passes	on each gating slice, the paired difference against the current build did not fall below –(the smallest difference the slice can see)
a failure is a bug	a failure is <i>helped · hurt · cannot tell</i> , per slice, with the interval beside it
flakiness is zero	the noise floor is measured and reported; a "cannot tell" on a gating slice blocks , because it means the slice is too small to protect
tests run on the diff	the suite runs on recorded fixtures first, then a smaller live smoke

The gate is only as good as its slices. A slice with six tickets protects nothing.

MEASURING AI SYSTEMS • PROBLEM 2

| Who judges the judge?

The problem: who judges the judge?

The output. Each triage result carries a one-sentence **rationale**: *"Refund of \$52.99 exceeds the \$50 cap; holding for approval."* Free text, so no rule can score it.

The judge. A second model call with one yes/no rubric question: *Does the rationale support the action taken?*

The check. Two people on the team label **thirty** rationales by hand, separately. Then the judge labels the same thirty.

What came back

- The two people **disagree on 5** of the 30.
- The judge disagrees with each of them on about **6**.

**Which of the three is right?
And is the judge good
enough to score the other
five hundred?**

The same question applies to a person: a new hire labelling tickets is also a judge.

Resolution: reliability, then validity

A scorer that is not exact match is a **rater**: a person or a model. An instrument, checked in this order:

- **Reliability** — the same answer twice? two raters agree?
- **Validity** — does it measure the thing meant?

$$\text{validity} \leq \sqrt{\text{reliability}}$$

THE OUTPUT IS	"TRUTH" IS	VALIDITY MEANS
factual — the action, the amount	a gold label	agreement with it, per slice
preferential — which reply is better	a human majority	agreement with the majority; same verdict when the pair is swapped
stylistic — friendly, on-brand	none; a panel <i>defines</i> it	agreement with the panel's adjudicated label, once the panel agrees with itself

Example: two humans, thirty labels

Person A against person B, same thirty rationales:

	B: YES	B: NO
A: yes	19	3
A: no	2	6

Raw agreement: 25 of 30 = **83%**

By chance alone (A says yes 22/30, B 21/30):

$$p_e = \frac{22}{30} \cdot \frac{21}{30} + \frac{8}{30} \cdot \frac{9}{30} = 0.59$$

$$\kappa = \frac{p_o - p_e}{1 - p_e} = \frac{0.83 - 0.59}{0.41} = 0.59$$

- $\kappa = 1$: perfect agreement · $\kappa = 0$: coin flips
- the 5 disagreements: **fix the rubric, then adjudicate**
- the adjudicated labels are the **reference**

p_o : observed agreement · p_e : agreement expected by chance from each rater's yes-rate.

Example: the judge against the reference

The judge against the thirty adjudicated labels:

	JUDGE: YES	JUDGE: NO
reference: yes	20	2
reference: no	4	4

Raw agreement 24 of 30 = 80% · $\kappa = 0.44$, 95% interval **0.05 to 0.84**

Thirty labels show a judge is not *broken*. They cannot show it is *good*.

Interval from κ 's standard error, $\sqrt{(p_o(1 - p_o) / (n(1 - p_e)^2))}$, about 0.20.

$$\text{sensitivity} = \frac{\text{caught}}{\text{real passes}} = \frac{20}{22} = 0.91$$

$$\text{specificity} = \frac{\text{flagged}}{\text{real failures}} = \frac{4}{8} = 0.50$$

- says yes to half the failures: **lenient**
- humans agree with each other at 0.59; the judge with them at 0.44
- **the humans are the ceiling**

Agreement measures

MEASURE	FOR	WHAT IT CORRECTS FOR	USE IT WHEN
percent agreement	any	nothing	never on its own: two lenient raters agree by accident
Cohen's κ	two raters, categorical labels	chance agreement from each rater's base rate	one judge against one person, or two people — the two examples just shown
Fleiss' κ	many raters, categorical	chance, across a panel	three or more labellers on the same items
Krippendorff's α	any number of raters, missing labels, nominal / ordinal / interval	chance, and partial credit for near-misses on ordered scales	a panel that did not all label every item; a 1–5 scale
intraclass correlation (ICC)	continuous scores	rater bias and scale use	scalar judge scores against scalar human scores
Spearman ρ / Kendall τ	rankings	nothing about chance; measures order	a judge that ranks candidates, against a human ranking

The judge's own reliability: run it three times on the same items; the spread is its noise floor, and no agreement number can beat it.

Disagreement is data: the items two people cannot agree on are the requirement, not yet defined.

Validity: factual and preferential outputs

Factual outputs — the action, the amount, "the total is under \$50"

- Score the judge against a **gold label**: the four counts, per slice
- Its **sensitivity** and **specificity** are what you need to correct the rates it reports
- Per slice, because a judge that is right on short tickets can be wrong on long ones

Preferences — "which of these two replies is better?"

- Agreement with a **human majority**, not one person
- **Swap the pair** and ask again: the share of verdicts that flip is position bias, measured
- **Length-matched pairs**: the share of wins that go to the longer reply, when the rubric says length is not the point

Validity: scalar scores, and tests for every judge

Scalar scores — "rate this 1–10"

- Correlation with human scores, and **calibration**: is the judge's 8 the humans' 8?
- Usually the wrong design: the judge's 7 and 8 mean nothing. Prefer yes/no questions, or pairs.

Whatever the output

- **Self-preference**: is the judge the same model family as the system under test? Get a second opinion from another.
- **Injection**: the text being judged was written by a model and can contain instructions. Does the verdict move when it does?
- **Rubric sensitivity**: reword the rubric; a robust judge gives the same verdicts.

MEASURING AI SYSTEMS • PROBLEM 2, HARDER

| What if there is no right answer?

The problem, harder: tone and style

The rationale had a right answer somewhere.

Tone does not. "Is this reply friendly?" has no gold label. Two people will disagree, and neither is wrong.

So the construct has to be **built** before it can be measured:

1. **Decompose** it into observable yes/no proxies a stranger could answer from the text: *names what the customer asked for · no blame language · one apology, not three · offers a next step*
2. **Anchor** any scale with examples: this is a 2, this is a 4

3. **Panel:** three people label; disagreements are adjudicated; the adjudicated label is the reference
4. **Measure the panel first:** κ or α among the humans. If they are at 0.3, the requirement is not defined yet, and no judge can be validated against it. Fix the rubric, not the judge.

Resolution: the judge against the panel

5. **The judge** against the adjudicated label, per proxy question, per slice
6. **Report the ceiling** beside the judge: *humans* $\alpha = 0.7 \cdot \text{judge}$ $\kappa = 0.6$

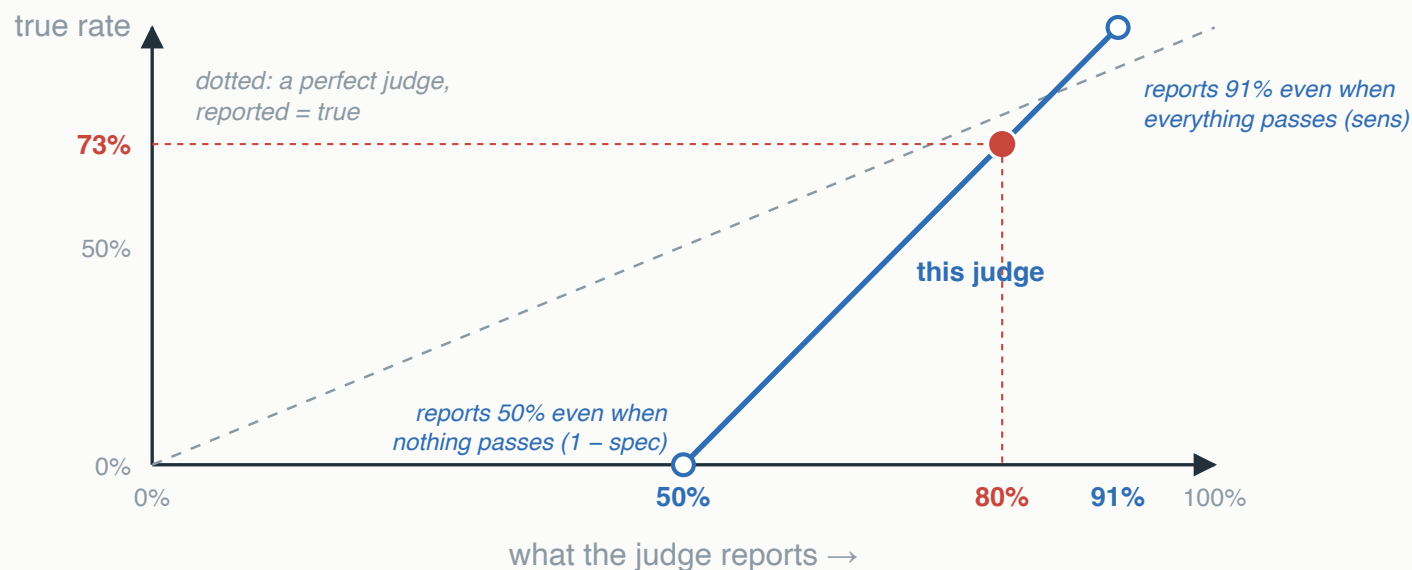
α : Krippendorff's alpha among the panel · κ : Cohen's kappa, judge against the adjudicated label.

- **What this buys:** a number for "friendly" two teams can argue about with the same rubric in hand
- **What it cannot:** whether the panel's idea of friendly is the customer's — that is a survey, not a harness

MEASURING AI SYSTEMS • PROBLEM 2, CONTINUED

| What is a judge's number worth?

Resolution: correcting a judge's rate



Judge measured on 30 labels:
catches 20 of 22 real passes (sens 0.91),
flags 4 of 8 real failures (spec 0.50).

$$\text{reported} = \text{sens} \cdot \text{true} + (1 - \text{spec})(1 - \text{true})$$

$$\text{true} = (\text{reported} - 0.50) \div 0.41$$

Reported 80% → about 73%.

Slope $1 \div 0.41 \approx 2.4$: every point the judge moves is 2.4 points of truth.

Rogan–Gladen (1978). A straight line, steeper the worse the judge; a lenient judge inflates every rate it reports.

Where to spend the labels

Human labels are the budget for truth. Spend them where they change a number.

- **Stratify by slice.** Agreement on the easy slice tells you nothing about the hard one.
- **Oversample failures.** Specificity needs real failures to be measured on; a random 30 has too few.
- **Label the disagreements.** Where the judge and the first labeller differ is where the rubric is unclear.

When to do it again

- The judge's **model changed** — the vendor updated it, or you switched
- The **rubric** changed
- The system under test changed *what kind* of text it produces (longer, more fluent, a new failure mode)
- **On a schedule**, regardless: a judge validated once, on a different model, is an opinion

A judge is a component with an error rate. It drifts like one.

The report row

One row per slice of the suite. Counts first, the paired difference with its interval, the smallest difference the slice could see, and the verdict.

SLICE	N	BEFORE	AFTER	PAIRED DIFF (95%)	CAN SEE	VERDICT
refund-over-cap	8	73%	85%	+12 (-7, +32)	32 pts	cannot tell
refund-under-cap	60	92%	95%	+3 (-4, +10)	11 pts	cannot tell
instruction-in-ticket	40	40%	80%	+40 (+22, +58)	14 pts	helped
no-money	60	97%	88%	-9 (-16, -2)	11 pts	hurt
<i>rationale supports action</i> (judge-scored)	60	81%	83%	+2 (-8, +12)	11 pts	cannot tell · judge κ 0.44 (0.05–0.84)

Every verdict is a verdict about a slice. The suite's total is not a row. A rate scored by a judge carries the judge's κ on the same line.

n: tickets on the slice · paired diff: mean per-ticket difference, after minus before, with its 95% interval · can see: the smallest difference this n could clear.

STEP 5 OF 7

Architecture, part 1

Where the model lives, and what it is allowed to decide.

Where we are: step 5

Step 5 is **high-level architecture**. Part 1, with three problems:

- **Where the model lives**: placement, the cost and latency arithmetic, capacity, the failure path
- **What the model is allowed to decide**: the determinism boundary, where correctness lives, where knowledge lives, the build
- **When the model cannot see everything**: the cascade, and the one shape it takes everywhere

#	STEP
1	Scope & requirements
2	Frame the AI task
3	Metrics
4	Data
5	Architecture · part 1
6	Deep dive
7	Failure, scale & operations

ARCHITECTURE • PROBLEM 1

| Where does the model live?

The problem: a million tickets a month

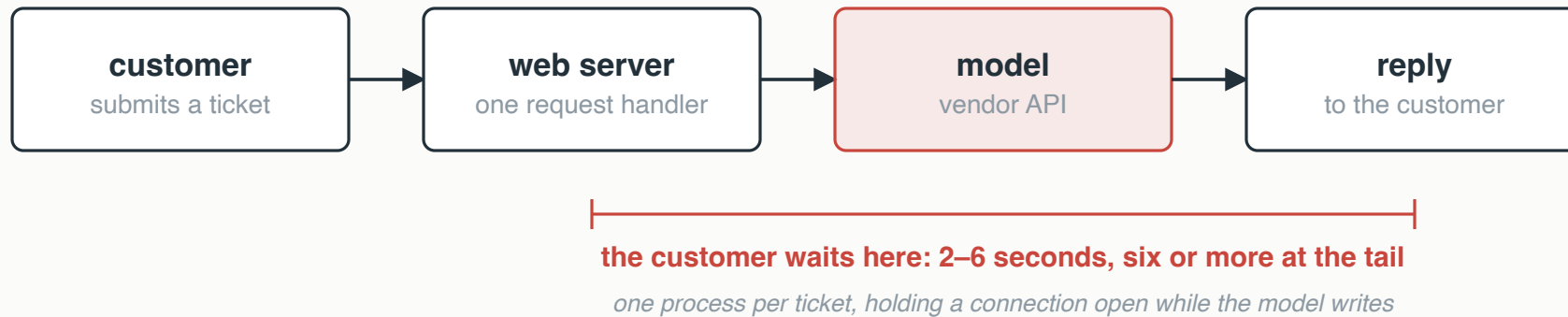
System: support-ticket triage. A web page where customers submit a ticket; the system answers within about a minute.

- **Input:** the ticket text and the customer's account
- **Model:** classify the ticket — answer · refund · hold · escalate
- **Output:** the action, a refund amount, a one-sentence rationale
- **One model call:** about 3,000 tokens in, 200 out · 2–6 seconds

REQUIREMENT	VALUE
scale	1,000,000 tickets a month
latency	acknowledgment instantly ; answer within a minute
cost	well under what a person costs per ticket

Where does the model call go? And can we afford it?

The obvious design



One process per ticket, one model call inside the request. Works at one ticket a minute.

Two models, two designs

Which model. Models come in sizes: a **small** one is cheap and fast; a **large** one is stronger, slower, and about 25× the price per token.

	PER MILLION TOKENS IN	OUT
small model	≈ \$0.10	≈ \$0.40
large model	≈ \$2.50	≈ \$10

Prices are September 2026 orders of magnitude. Design 2 is introduced only as a count of model calls; how such chains are built is a later lecture.

Design 1 — one call per ticket. Classify and answer at once.

≈ 3,200 tokens per ticket.

Design 2 — four calls per ticket. Classify → look up → draft → check. Each call sees everything before it, so the context grows.

≈ 20,000 tokens per ticket.

AT A MILLION TICKETS A MONTH • FOUR AXES

| cost • latency • capacity • tail latency

Cost

$$\text{cost per month} = \text{tickets} \times (\text{tokens}_{\text{in}} \cdot p_{\text{in}} + \text{tokens}_{\text{out}} \cdot p_{\text{out}})$$

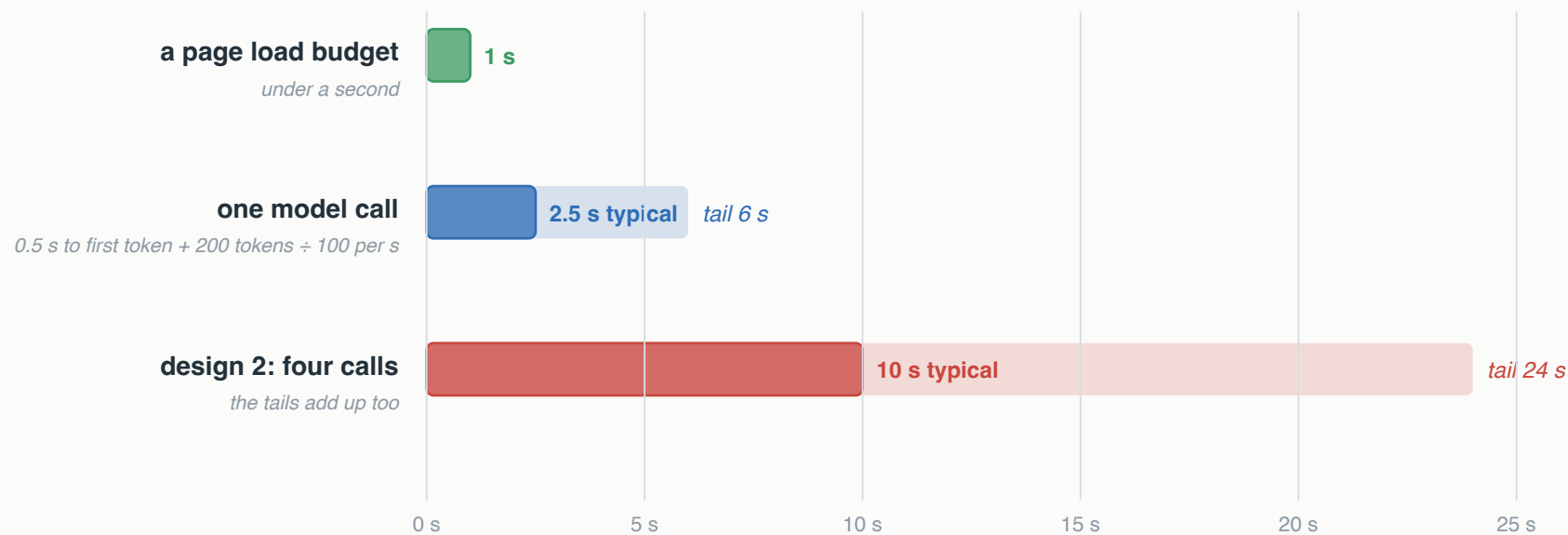
which design	which model		
	small · cheap	large · strong	
design 1 one call per ticket	≈ \$380 \$0.0004 a ticket <i>a month, for a million tickets</i>	≈ \$9,500 \$0.01 a ticket <i>a month, for a million tickets</i>	× 25 for the model
design 2 four calls per ticket	≈ \$2,300 \$0.002 a ticket <i>a month, for a million tickets</i>	≈ \$58,000 \$0.06 a ticket <i>a month, for a million tickets</i>	× 150 for both
× 6 for design 2	<i>prices per million tokens, September 2026, order of magnitude: small \$0.10 in / \$0.40 out · large \$2.50 in / \$10 out design 2: classify → look up → draft → check; the context grows each call, so four calls send about 20,000 tokens</i>		

A model size and a call count: two choices nobody calls architecture, and a 150x spread in the bill.

p_{in} , p_{out} : price per token in and out · top left, worked: $1\text{M} \times (3,000 \times \$0.10/\text{M} + 200 \times \$0.40/\text{M}) = \$300 + \80

Latency

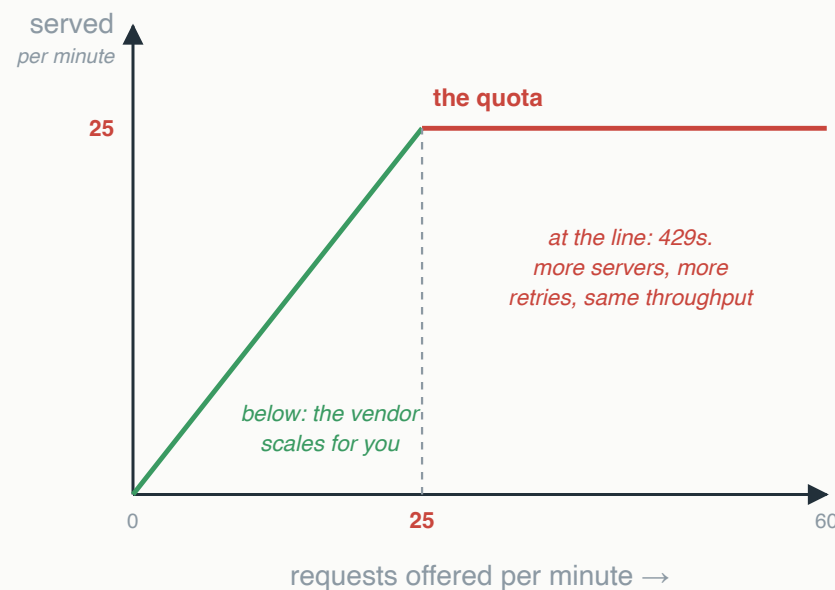
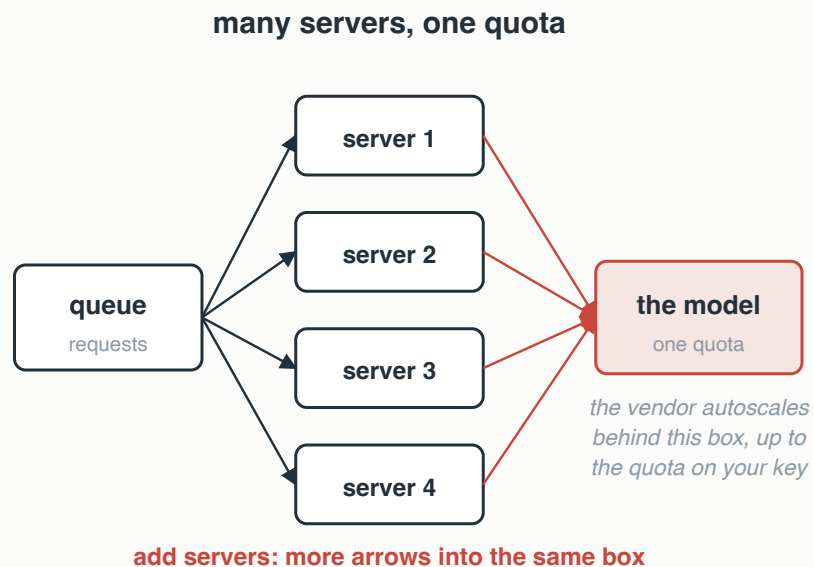
$$\text{latency per ticket} \approx \text{calls} \times \left(t_{\text{first token}} + \frac{\text{tokens}_{\text{out}}}{\text{rate}} \right)$$



What the customer waits for if the model is inside the request. The page budget is the thin green bar.

$t_{\text{first token}}$: the pause before the model starts writing, about 0.5 s · rate: about 100 tokens a second for a hosted model · orders of magnitude only.

Capacity

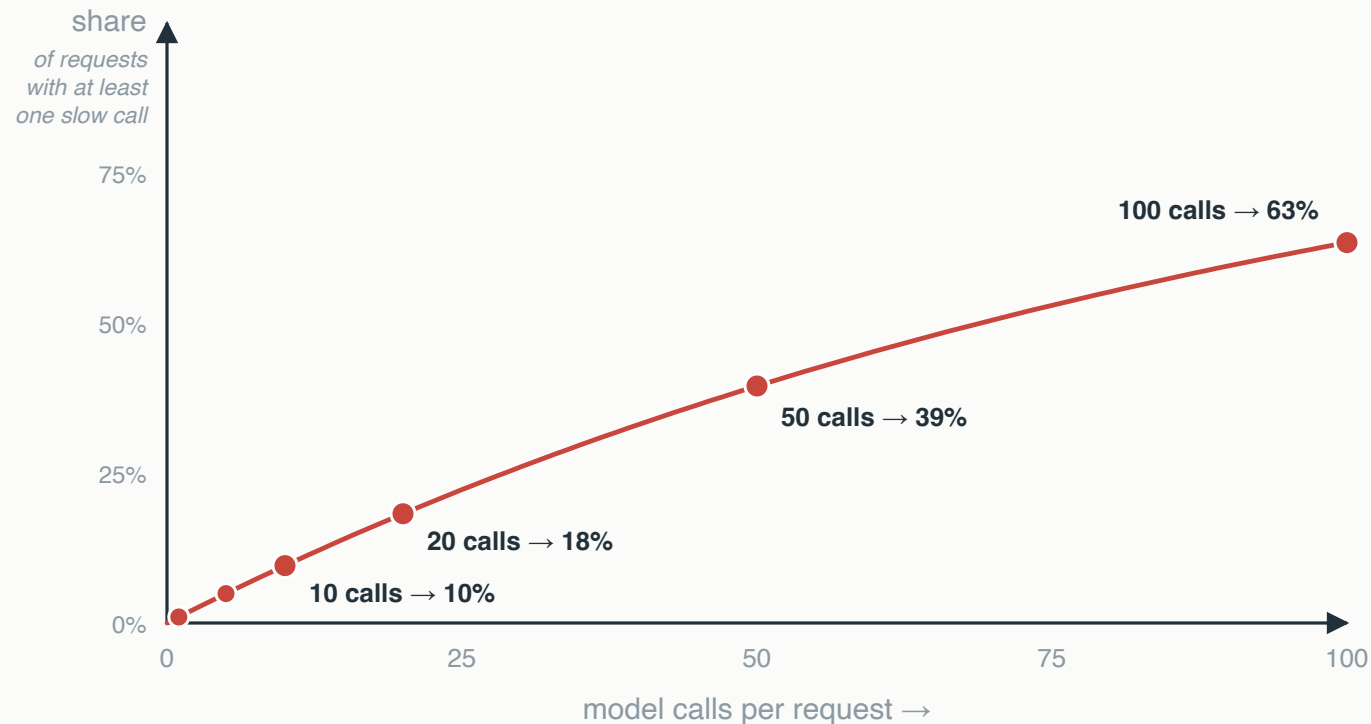


A vendor key at 25 requests a minute. Below the line classic scaling holds; at it, nothing you add on your side helps.

$$\text{capacity} = 25 \frac{\text{req}}{\text{min}} \times 60 \times 24 \times 30 \approx 1,080,000 \frac{\text{req}}{\text{month}}$$

quota: the vendor's limit per key on requests or tokens per minute; 25 a minute is an illustrative value. A million tickets against 1.08 million of quota is no headroom.

Tail latency



One call in a hundred is slow.

A request that makes ten model calls in sequence is slow one time in ten.

The tail of one call becomes the median of the system.

Dean & Barroso, The Tail at Scale, with the fan-out replaced by a chain of calls. $1 - 0.99^k$; one call $\rightarrow 1\%$, five $\rightarrow 5\%$.

$P(\text{at least one slow call}) = 1 - 0.99^k$, where k is the number of model calls a request makes in sequence and one call in a hundred is slow.

A model as a component

A model in a system design is:

- **slow** — seconds, not milliseconds; one token at a time
- **expensive** — priced per token
- **capacity-limited** — a quota, not your servers
- **nondeterministic** — a retry is a new sample
- **partly not yours** — a vendor can change it

Where a model lives, and how it is invoked, is the dominant decision in an AI system architecture.

Five placements

PLACEMENT	THE MODEL RUNS...	LATENCY TO THE USER	FRESHNESS	COST SHAPE	WHAT ISOLATES A FAILURE
Precompute / batch	on everything, on a schedule; results stored	none — a lookup	stale by the schedule	cheapest per item; you pay for items nobody asks for	the last good batch is still there
Online, synchronous	per request, the user waits	the model's full latency, in the path	current	per request, at peak	nothing: a slow model is a slow page
Streaming	per request, output shown as it is produced	time to first token	current	per request	the user sees a partial answer
Asynchronous / queued	per request, after an acknowledgment	seconds to minutes, off the path	current	per request, smoothed by the queue	the queue absorbs a slow or dead model
On-device / edge	on the user's hardware	milliseconds	as fresh as the last update	free per call; expensive to ship	the network is not involved

The obvious design is row two. Most systems use **two rows**: precompute what you can, and put a model online only for what you cannot.

Choosing a placement

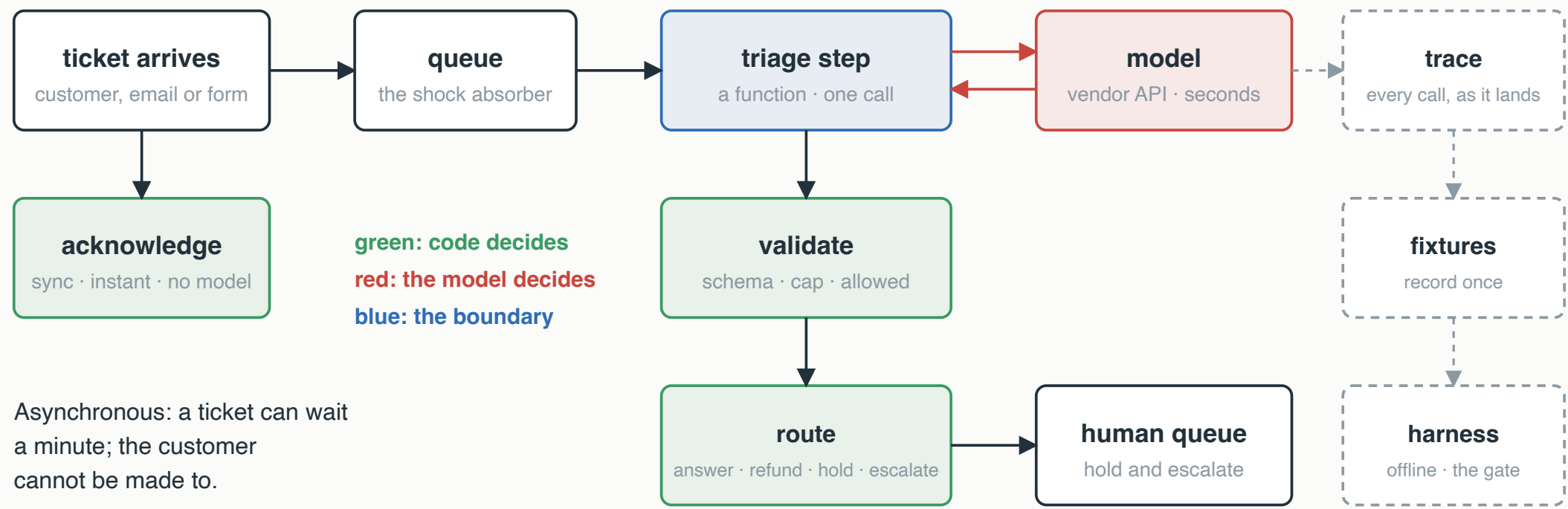
In this order:

- 1. **What must be instant?** No model in that path.
- 2. **What can be streamed?**
- 3. **What can be deferred?** Queue it.
- 4. **What can be done before anyone asks?**
Precompute it.

The model is synchronous only when none of the above applies; then the latency budget picks the model.

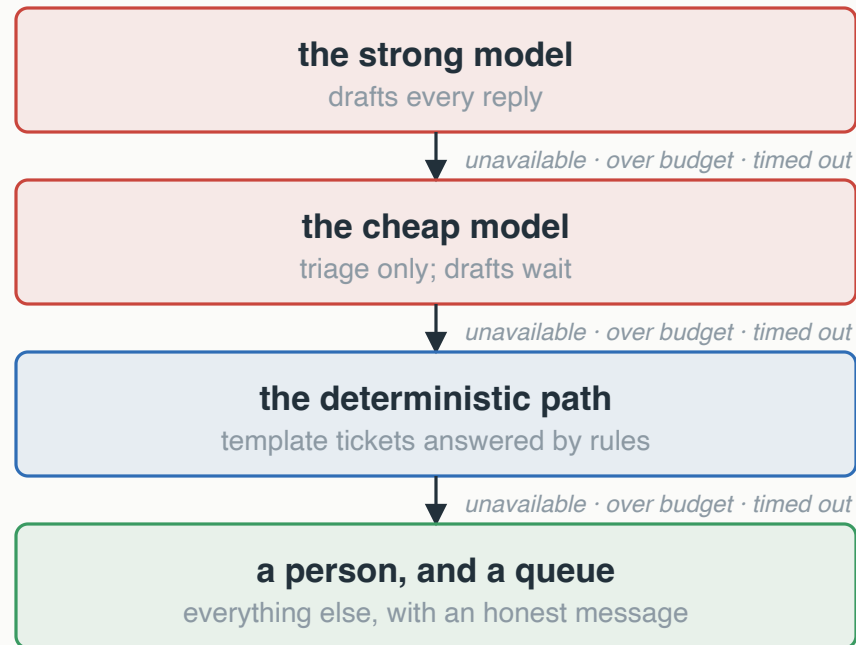
SYSTEM	PLACEMENT	WHY
a fraud score in a card authorization	sync, small model, features precomputed	tens of milliseconds
a drafted reply an agent will read	streamed	the first token matters most
a support ticket	async: acknowledge, queue, answer	a minute is fine; the customer cannot be made to wait

Resolution: the triage system



Nothing model-shaped is in a synchronous path; the harness sits offline, on the same fixtures.

Resolution, continued: the failure path



Each rung has its own evaluation.

The bottom rung is exercised on purpose, on a schedule, so it works when it is needed.

The bottom of every ladder is a human and a queue. Design the queue: its depth, who works it, and what the customer is told.

The degradation ladder. What runs at 9 a.m. when the model is down is a design decision, or it is a blank page.

The trade space

Four axes: **latency** · **quality** · **cost** · **freshness**.

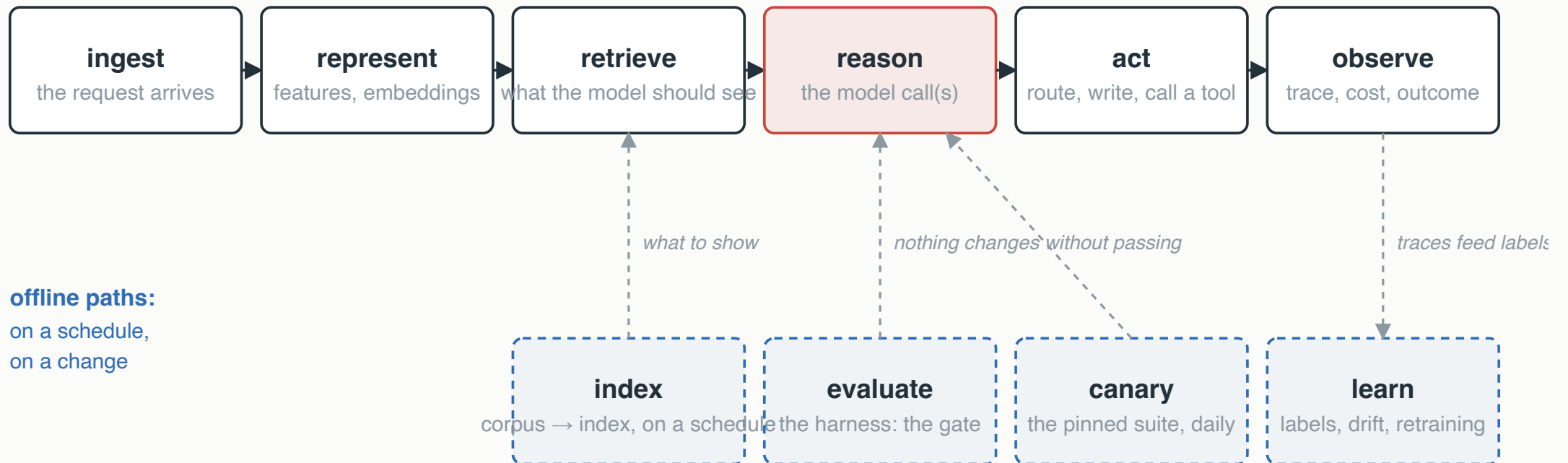
LEVER	BETTER	WORSE
a bigger model	quality	latency, cost
precompute	latency, cost	freshness
a cache	latency, cost	freshness; correctness if "similar" ≠ "same"
more retrieval	quality	latency, cost
more calls	quality, sometimes	latency, cost, always
a cheaper model first	cost	quality on what it gets wrong

You choose a point. The architecture is that point.

- the requirements table is where the point is written down
- the architecture is where it becomes components
- a design that has not said which axis gives way has not decided

A reference architecture

online path: one request



Not every system has every box. A design should say which it has, where the model is, and where the harness and the trace sit.

| What is the model allowed to decide?

The problem: what may the model decide?

The triage step's prompt says: *refunds above \$50 need approval; choose hold.*

One ticket asks for **\$52.99**. Thirty runs, nothing changed between them:

THE MODEL CHOSE	TIMES
hold	25
refund \$50.00	3
refund \$52.99	2

Two runs in thirty, the rule in the prompt was ignored and money moved.

- **The support bot** from the failure of the week stated a policy nobody had written
- Same shape: **a decision with a right answer was left to the model**, and the model produced a plausible one

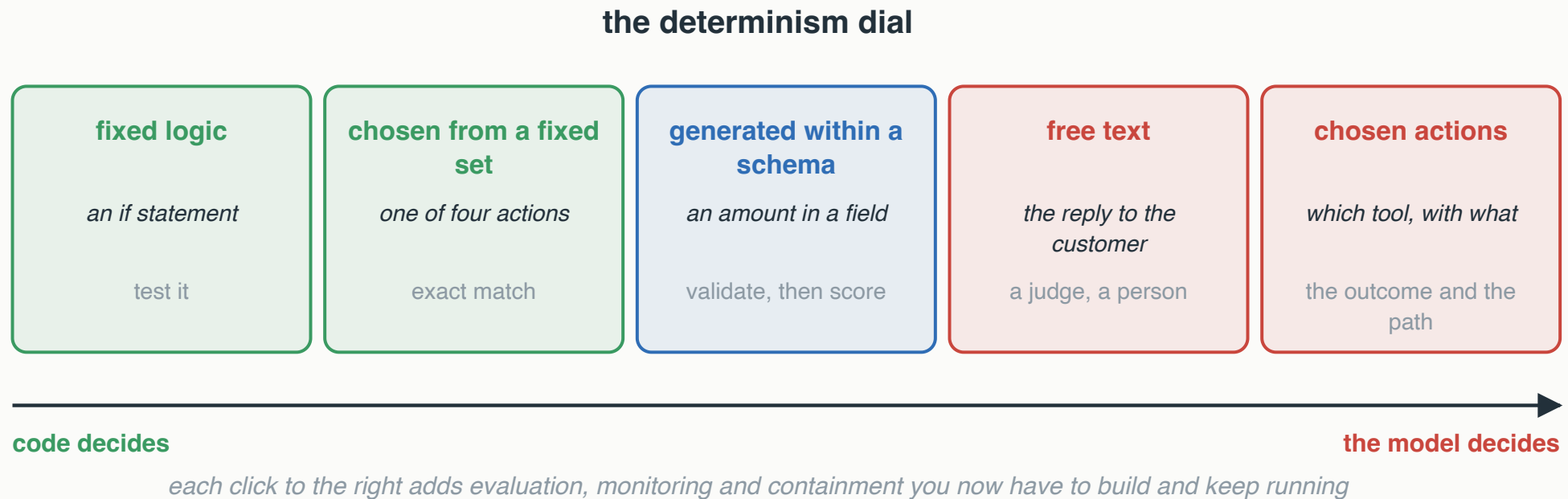
Which decisions belong to the model, which to code, and who is responsible for correctness?

Example: every decision in the triage step

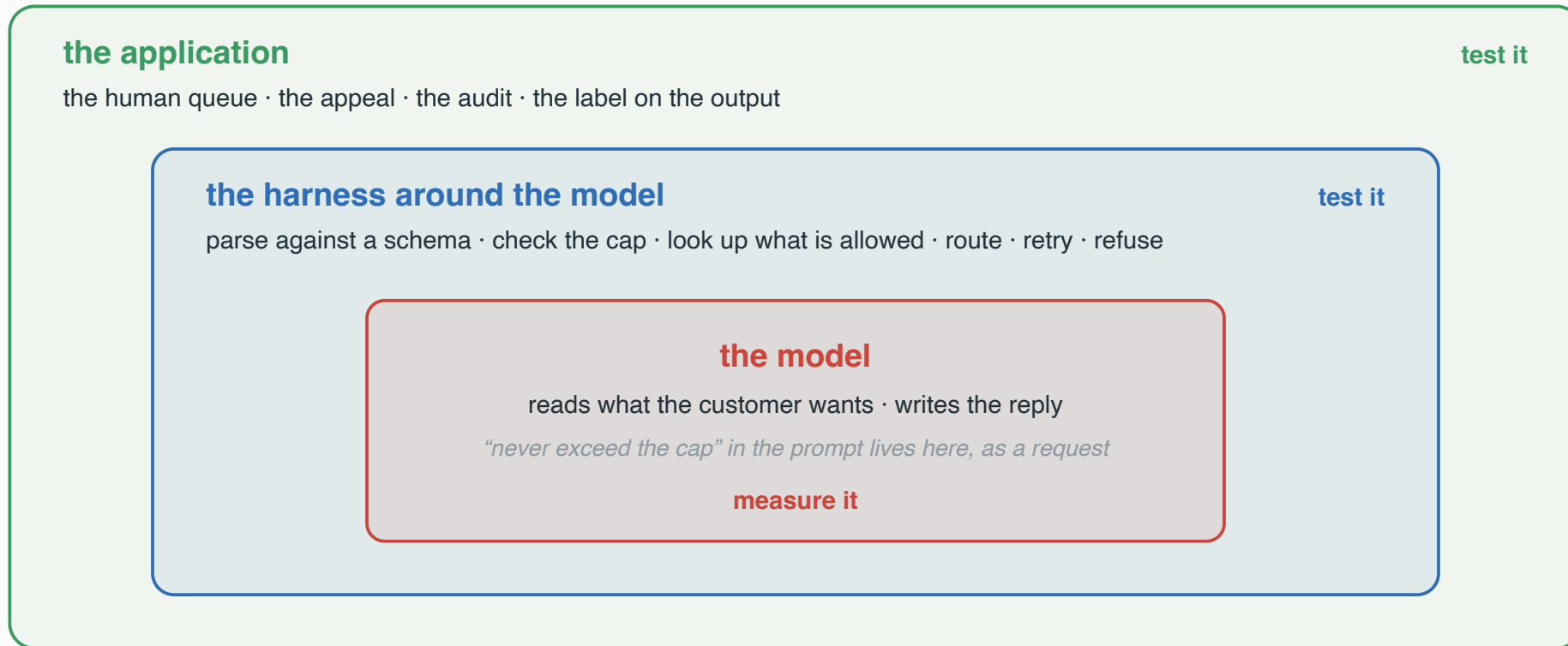
DECISION	WHERE IT LIVES TODAY	WHO SHOULD DECIDE	HOW YOU CHECK IT
which action: answer · refund · hold · escalate	the model	the model — reading what the customer wants is the job	exact match against the golden set, per slice
the refund amount	the model	the model proposes	distance from the right amount
is the amount under the cap?	a sentence in the prompt	code — one if	a unit test
is a refund allowed on this account?	the model's guess	code — a lookup	a unit test
what the policy actually says	the model's memory	a lookup or a retrieved document	is the document current?
the wording of the reply	the model	the model	a judge, validated
a third complaint from the same customer escalates	nobody	code — a count	a unit test
which step runs next, when the steps are known	the model, in "agent" designs	code — a state machine	a unit test

Six of eight rows moved out of the model. **Every row that moves out is one you never have to evaluate, monitor, or defend.**

Resolution: the determinism dial



Resolution, continued: where correctness lives



A rule stated inside the red box is a request. A rule enforced in the blue box is a rule. Correctness lives in the layer that can enforce it.

Inside the red box you **measure**. Outside it you **test**. The \$52.99 refund is what a request looks like two times in thirty.

Resolution, continued: across the boundary

Out of the model, into code

- parsing and formatting
- arithmetic
- lookups — accounts, prices, dates, policy text
- policy checks — caps, scopes, allowed actions
- routing on a known key
- retries, timeouts, idempotency
- *which step comes next*, when the steps are known

Into the model

- judgment under ambiguity: what does this customer actually want?
- language: reading it, writing it
- an input space too large to enumerate

Every capability you move out is one you never have to evaluate, monitor, or defend.

◆ *The best agent is often the one you replaced with a state machine. Nobody writes a blog post about that one.*

Where knowledge lives

RUNG	CHANGE IT BY	FRESHNESS	INVALIDATE?
in the weights	retraining, fine-tuning	frozen at training	no
in the instructions	editing a prompt	at deploy	yes
in examples	editing a prompt	at deploy	yes
retrieved per request	updating a store	per request	yes
looked up by a tool	the source of truth	live	n/a

- a fact that changes weekly does not belong in the weights
- policy belongs where it can be **read, versioned, invalidated**: retrieved, or in code
- a price belongs in a lookup
- the support bot had policy on no rung; the model filled the gap with the most plausible sentence

The build

release 2026-09-29-b		one manifest
code	commit 4f2c...	
prompts and templates	triage-policy v7	
model identifier	gemini-3.1-flash-lite (+ the large tier)	
index snapshot	runbooks 2026-09-21 · hash 9a1e...	
tool definitions	lookup_account v2 · issue_refund v3	
thresholds, caps, budgets	cap \$50 · escalate \$200 · 25 req/min	
evaluation suite + noise floor	golden v12 · floor 8 pts	

Any line changes → a new release → through the gate.

A prompt edit without the suite: the system changed, nobody measured.

The vendor changes the model: no line changed, the system did. That is the canary.

"Roll back" = to which row set?

The build is every line, versioned together. Values illustrative.

Build or rent

- **Rent** the model: better than you could train, on someone else's capacity; they can change it, and your quota is your ceiling
- **Own** the harness, the evaluation suite, the golden set, the traces, the prompts, the index: your specification

PORTABLE	NOT
the golden set and scorers	a prompt tuned to one model's habits
the harness and the traces	a fine-tune
the index and the corpus	vendor-specific tool schemas
the boundary and the code outside it	anything inside the boundary that leans on one model

The boundary is also a **lock-in map**: the more outside it, the less a vendor holds.

| What if the model cannot see everything?

The problem: a billion candidates

System: a short-video feed.

- hundreds of millions of people a day
- a request: the next **ten videos** for one person, from **billions**
- budget: **about 300 ms** end to end
- the best model: scores one (person, video) pair in **about 1 ms**, batched

The obvious design: score every video, take the top ten.

CANDIDATES SCORED	TIME PER REQUEST
1,000,000,000	11 days
1,000,000	17 minutes
10,000	10 s
500	tens of ms , batched

The strong model can see a few hundred. Who chooses them?

Resolution: the cascade

- score every candidate with something cheap, keep thousands
- score those with something better, keep hundreds
- the strong model sees the hundreds
- select ten by rule

$$\text{cost} = N \cdot c_{\text{cheap}} + k \cdot c_{\text{strong}}, \quad k \ll N$$

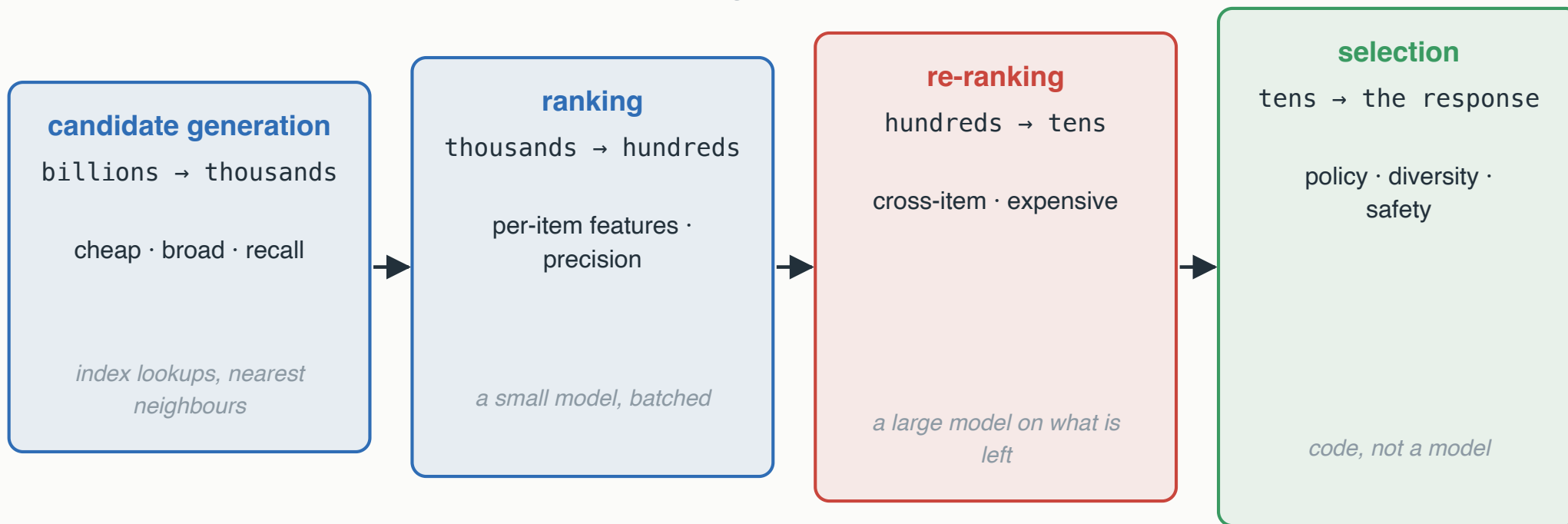
N : candidates in the pool · k : candidates the strong model sees · c : cost per candidate for each model.

Each stage's ceiling is the recall of the stage before it.

A video the first stage never surfaces does not exist, however good the ranker is.

Resolution, continued: the stages

cost per item rises left to right; the number of items falls faster



One shape, many names. Each stage's ceiling is the recall of the stage before it.

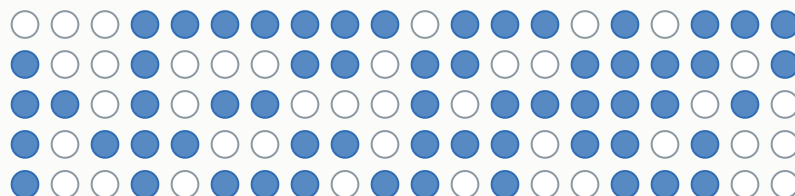
One architecture, many names

CALLED	CANDIDATES COME FROM	RANKED BY	SELECTED BY	THE STRONG MODEL DOES
a recommender	followed, similar, trending, nearest-neighbour indices	a multi-task model: watch, like, share, skip	diversity, integrity, freshness rules	re-rank a few hundred
search	a lexical index and a dense index	a learned ranker over query-document features	policy, dedup, freshness	re-rank the top tens
retrieval for a language model	lexical + dense over the corpus	a reranker	access control, a token budget	<i>read</i> the top five and write
model routing	the models you run	difficulty or uncertainty of the request	budget and policy	answer only what the cheap model could not
tool selection	the tools a policy allows	the model's own ranking over descriptions	the authorization layer	pick one and call it

Learn it once. The payload changes; the shape does not. And in every row, the last column is a **model choosing among what code allowed**.

The recall ceiling

the 100 items a labelled set says are right for this request



filled: surfaced by candidate generation (60) · open: never surfaced (40)

recall, per stage

candidate generation



recall@1000 of the labelled set: 60 of 100 surfaced

reranker, on what it was given



ranks 57 of the 60 into the top ten: 95% of what it saw

the system, end to end



57 of 100: the stage before is the ceiling

A ten-point better reranker moves the middle bar. It cannot move the bottom one past the top one.

- measure the stages **separately**: candidate generation as **recall@k** against a labelled set, per slice; ranking on what it was given; selection on what policy removed
 - each stage's contract is a **recall floor**, not a correctness guarantee
- recall@k: the share of labelled right answers anywhere in a stage's top k.

The architecture checklist

	THE DESIGN SAYS
Placement	where the model lives — batch, sync, streaming, async, edge — and <i>why</i> , from the three numbers
Sync / async	which paths have a model in them and which must not; what the user is waiting for
The arithmetic	tickets × tokens × price; calls × context; the quota against the scale; the latency of the chain
The boundary	what code decides, what the model decides; the dial position for each output
The cascade	where cost demands one: stages, candidates per stage, a recall floor per stage
The failure path	what runs when the model is unavailable or over budget; the human at the bottom
The build	what is versioned together; where the harness and the trace sit
One rejected alternative	a placement you did not choose, and the reason

A single box labelled "AI" is level one. Boxes with no placement reasoning is level two.

Which step did they skip?

Step 5. The model lived in the synchronous path to the customer with nothing between them — no validation, no person, no label.

Step 2. The model was allowed to decide *policy*. Policy is a lookup. It was on no rung of the ladder the model could reach.

Step 7. There was no failure path. "I don't know, a person will reply" was not an output the system could produce.

What the fix looked like — every item a box, not a prompt:

- policy moved **out** of the model, into something it reads
 - a **person** between the model and anything irreversible
 - a **label** on every model-written reply
- ◆ *They did get one thing right: the bot's name was easy to search for.*

Design the support desk

Customers submit tickets through a web form: the order, a free-text message. Two models:

- **A classifier**, trained, a black box: a **score 0–1** for "money is involved," in milliseconds
- **A language model**, the triage step: an **action** · a **refund amount** · a **rationale** · the reply, in seconds
- **Scale**: 50,000 a month; a bad Monday is ten times an ordinary hour
- **The customer**: a confirmation now, an answer within an hour

Task: the three numbers · the architecture: web server, databases, the two models, the review queue, with tables and sync/async per hop · the failure path, per model, if time.

Which model runs before the queue, which after?

Where does the refund cap live?

What runs when each model is down?

Put the model behind a URL

The question. The triage step has been a function call in your process. Move it behind a URL in your Learner Lab and measure what changed: **cold start • warm p95 • a time limit.**

0 — The arithmetic. A million tickets a month, on paper, before anything is deployed.

1 — Run it here. `src/function/handler.py` as a plain function, then on a local URL. Same harness, same report.

2 — Deploy. `./deploy.sh`: the package for Lambda's Linux, the lab role, a URL, the key in the function.

3 — Call it. Three runs over the URL, scored against the local baseline. **Nothing changed.**

4 — Measure. `latency.py`: the cold start split into import, model and the rest; warm p50 and p95. `--recycle` forces a cold start.

5 — Kill it. `--timeout 1`, below the model's own latency: every call comes back as `error`. Then set the limit from your p95. What is in a dead call's record, and what should the system do next?

p50, p95: the median and the 95th percentile of the round trip.

Before you go

- **Reading for next week:** Sambasivan et al., *Everyone wants to do the model work, not the data work: Data Cascades* (2021) · Breck et al., *The ML Test Score* (2017)
 - **Project 1** continues in your repository. The comparison you owe — helped, hurt, cannot tell, per slice — is exactly the paired difference from tonight, with the smallest detectable difference beside it. The judge's four counts go on the same line as any rate the judge produced.
 - Next week: data as specification, and retrieval — the cascade, again, with documents in it.
- ◆ *Before you leave: name one thing in your current system that the model decides and a lookup could. That's your first architecture review.*